Classical Macro-Econometric Nowcasting Models

BVAR, DFM, MIDAS, and MIDAS-ML — with an application to Trinidad and Tobago's Real GDP Growth

Jose Luis Saboin, PhD
Inter-American Development Bank

GDP Growth Nowcasting Workshop: Trinidad and Tobago

October 27, 2026

"Four pre-machine-learning workhorses — still indispensable for nowcasting GDP."

Roadmap

- **Part I** — The mixed-frequency nowcasting problem
- **Part II** — Dynamic Factor Model (DFM)
- **Part III** — Bayesian VAR (BVAR)
- **Part IV** — MIDAS regression
- **Part V** — MIDAS-ML (sparse-group LASSO)
- **Part VI** — Bringing it together

Each part ends with the R package call that implements the model.

Part I — The mixed-frequency nowcasting problem

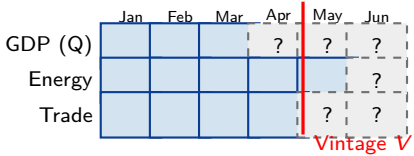
Why nowcasting is hard: ragged-edge information

GDP is released **quarterly**, about **six months** after the quarter ends.

But every month, fresh indicators arrive:

- Energy production and exports, cement and vehicle sales
- Card payments, credit, tax revenue, trade
- Google Trends, satellite night lights

We need a model that can **see today's monthly data** and produce a sensible nowcast of the **current-quarter GDP** *before the official release*.



Filled boxes = observed; dashed = missing.

Why classical models alongside the ML zoo?

The classical models bring:

- **Interpretability** — coefficients map to economic stories
- **Theory-consistent shrinkage** (Bayesian priors)
- **Latent state** — a single "business cycle factor"
- **Mixed-frequency** handled natively
- Small-sample behaviour is well understood

The ML zoo brings:

- **Nonlinearity** (trees, GBT, RF, LSTM)
- **Variable selection** from hundreds of indicators
- Strong empirical performance in short horizons
- No distributional assumptions

Together:
an ensemble that can pool both. The Trinidad and Tobago pipeline weights every model by its past accuracy, so a classical estimator joins on the same terms as any other.

Notation we will use throughout

- $y_q \in \mathbb{R}$ — target quarterly variable (real GDP growth, $Q = 1, \ldots, T_q$)
- $\mathbf{x}_t \in \mathbb{R}^N$ — high-frequency (monthly) indicators, $t = 1, \ldots, T$
- V — nowcast *vintage*: the calendar time at which the nowcast is produced
- $\mathcal{I}_V$ — information set available at vintage V (some series ragged-edged)
- h — forecast horizon (typically $h \in \{-2, -1, 0, +1, +2\}$ quarters relative to V)
- f_t — latent factor (DFM) ; $\mathbf{\Lambda}$ — factor loadings
- β — regression / VAR coefficients

Each model is a different answer to: "how do I combine monthly $\mathbf{x}_t$ into a quarterly $\hat{y}_q$?"

The four models in one line each

Model	Mechanism
DFM	Compress hundreds of indicators into a few latent factors; project GDP on those factors.
BVAR	Model $(y_q, \mathbf{x}_t)$ jointly as a VAR; shrink toward random-walk via a Minnesota prior.
MIDAS	Regress quarterly y on monthly $\mathbf{x}$ directly, using a *lag polynomial* to keep parameters few.
MIDAS-ML	Same as MIDAS, but pick variables via sparse-group LASSO when N is large.

Same target y_q. Same indicator pool. Different bets on the data-generating process.

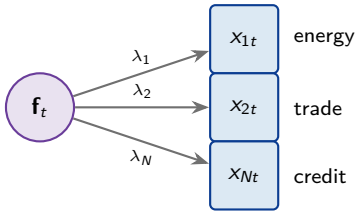
Part II — Dynamic Factor Model (DFM)

Factor model: the core idea

Hundreds of indicators all reflect the same underlying business cycle.

$$\underbrace{\mathbf{x}_t}_{N\times 1} = \underbrace{\mathbf{\Lambda}}_{N\times r}\underbrace{\mathbf{f}_t}_{r\times 1} + \underbrace{\varepsilon_t}_{N\times 1}$$

where:
$\mathbf{f}_t = r$ unobserved **common factors** (with $r \ll N$); $\mathbf{\Lambda}$ = factor loadings; ε_t = idiosyncratic noise (uncorrelated across series).



Why “dynamic”?

The factor itself **moves through time according to a VAR**:

$$\mathbf{f}_t \;=\; \mathbf{A}_1\mathbf{f}_{t-1} + \mathbf{A}_2\mathbf{f}_{t-2} + \cdots + \boldsymbol{\eta}_t$$

This gives the model two layers:

- **Measurement equation:** $\mathbf{x}_t = \boldsymbol{\Lambda}\mathbf{f}_t + \boldsymbol{\varepsilon}_t$
- **State equation:** $\mathbf{f}_t = \mathbf{A}_1\mathbf{f}_{t-1} + \cdots + \boldsymbol{\eta}_t$

This is a **state-space model** — the natural home for Kalman-filter machinery.

Intuition: a few latent “cycle” shocks propagate to every observable indicator; the data tell us the shocks via the Kalman filter.

DFM estimation in three steps

1. **Initial guess:** run **PCA** on the standardised indicators to get $\hat{\mathbf{\Lambda}}$ and $\hat{\mathbf{f}}_t$.
   - Fast, works even when data are stationary and balanced.
2. **Refine:** run the **EM algorithm** alternating
   - E-step: Kalman-smoother for $\mathbf{f}_t \mid \mathbf{x}_{1:T}$
   - M-step: re-estimate $\mathbf{\Lambda}, \mathbf{A}_i$, covariance matrices by OLS

   Iterate until likelihood converges.
3. **Nowcast:** fill missing $x_{j,t}$ in the ragged edge via the Kalman filter; project quarterly GDP on the smoothed $\hat{\mathbf{f}}_t$.

Most DFM packages (e.g. `nowcastDFM`) hide all this behind a single `dfm()` call.

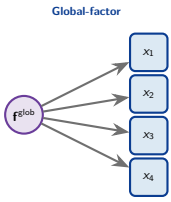
Blocks: one global factor vs. sectoral factors

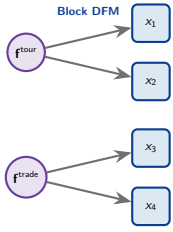
Global-factor DFM (default for small samples):

- Single factor explains everything
- Few parameters — safe with $T \approx 50$ quarters
- The safe default on a panel of about 50 quarters (`blocks <- blocks[1]`)

Multi-block DFM:

- Separate factors per sector (energy, fiscal, monetary. . .)
- Variables load only on their block's factor
- More flexible but needs T comfortably > 100





Mixed frequency without pain

Trick: treat the quarterly GDP as a **monthly latent variable observed every third month**.

$$y_q \;=\; (\text{average or aggregation of})\; y_t^{\text{mthly}}, \qquad t \in \text{quarter } q$$

Then the entire system runs at **monthly frequency**, with the Kalman filter handling the unobserved months automatically.

Why this matters in practice:

- No need to aggregate monthly indicators (and lose information)
- Information from *this* month's indicators feeds the nowcast *immediately*
- Ragged-edge handled by the same Kalman filter that estimates the factors

The R code: nowcastDFM

```r
library(nowcastDFM)

# One group of indicators (a specification) and the processed monthly panel
blocks <- blocks[1]  # one global factor (small sample)
output <- dfm(
  data         = train_data,
  blocks       = blocks,
  p            = 1,           # VAR order on the factor
  max_iter     = 1500,
  threshold    = 1e-5
)
nowcast <- predict(output, newdata = test_data)$X_smooth[, "GDP"]
```

- Takes a group of indicators as the Python pipeline defines it (one specification)
- Writes the same forecast and performance tables as the Python estimators
- So it can join the pool on equal terms, weighted by its own past accuracy

Part III — Bayesian VAR (BVAR)

Quick refresher: the VAR

A **vector autoregression** of order p stacks every observable into one big regression on its own past:

$$\mathbf{z}_t = \mathbf{A}_1 \mathbf{z}_{t-1} + \mathbf{A}_2 \mathbf{z}_{t-2} + \cdots + \mathbf{A}_p \mathbf{z}_{t-p} + \mathbf{u}_t, \qquad \mathbf{z}_t = (y_q, x_{1t}, \ldots, x_{Kt})'$$

Counting parameters:

For K variables and p lags: $\boxed{K^2 p}$ coefficients plus $K(K+1)/2$ covariance terms.

- $K = 8$, $p = 4 \;\rightarrow\; 256$ parameters
- With $T_q \approx 50$ quarterly observations, OLS will **overfit catastrophically**

Solution: pull the estimates toward a sensible prior.

Why a Bayesian prior fixes this

Instead of trusting OLS to choose all K^2p coefficients, we **tell the model what reasonable looks like**:

$$p(\beta) \propto \text{narrow Gaussian centred on a } \textit{random walk}$$

The Minnesota prior (Litterman 1980):

- Own first lag $\sim \mathcal{N}(1, \sigma_1^2)$ ($\Rightarrow$ "each series follows a RW unless data disagree")
- Own higher lags $\sim \mathcal{N}(0, \sigma_2^2)$ shrinking toward zero
- Cross-variable coefficients $\sim \mathcal{N}(0, \sigma_3^2)$ even more shrunk
- Hyper-parameter λ controls overall tightness (small $\lambda \Rightarrow$ strong shrinkage)

Posterior = **prior** $\times$ **likelihood** (Bayes' rule) — closed form for the Minnesota prior.

Mixed-frequency MF-BVAR

Same trick as the DFM: write the system at **monthly frequency** and let quarterly GDP be an unobserved monthly state with periodic observation.

$$\mathbf{z}_t^{\text{mthly}} = \mathbf{A}_1 \mathbf{z}_{t-1}^{\text{mthly}} + \cdots + \mathbf{u}_t$$

$$y_q \;=\; \sum_{t \in q} w_t \, y_t^{\text{mthly}} \quad \text{(e.g. } w_t = 1/3 \text{ for averaging)}$$

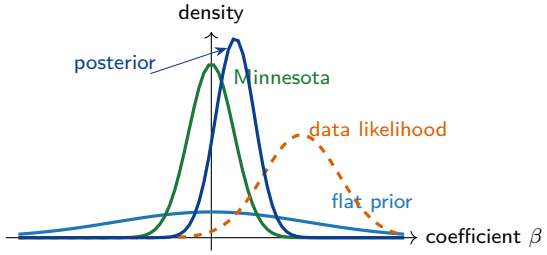
Estimation:

Gibbs sampler alternating

- Draw missing monthly y_t given current VAR and observed data
- Draw VAR coefficients given complete data and Minnesota prior

After burn-in, you have the **joint posterior** of parameters *and* the unobserved months.

What “shrinkage” looks like



Posterior (dark blue) = Minnesota prior × data likelihood (Bayes' rule). It sits between the two but much closer to the tighter Minnesota — the noisy OLS estimate at $+1.4$ gets shrunk to $\approx +0.37$.

The R code: `mfbvar`

```r
library(mfbvar)

# Variable ordering: monthly indicators first, quarterly GDP last
prior <- set_prior(
  Y           = train_data,
  freq        = c(rep("m", K-1), "q"),
  n_lags      = 4,
  n_burnin    = 1000,
  n_reps      = 5000,
  lambda1     = 0.2,          # Minnesota tightness
  prior_nu    = K + 2
)
fit      <- estimate_mfbvar(prior, prior = "minn")
nowcast <- predict(fit, newdata = test_data)$Y_q[, "GDP"]
```

- K is the size of the group of indicators (one specification of the pipeline)
- $\lambda_1 = 0.2$ is a common choice for small-sample emerging markets
- Same output tables as the other estimators

BVAR: strengths and limits for Trinidad and Tobago

Strengths

- **Joint distribution** of all variables — can answer "conditional on gas output, what's GDP?"
- Built-in shrinkage rescues a sample where $K^2 p > T$
- Native mixed-frequency
- Posterior gives natural uncertainty bands

Limits

- Number of variables capped — $K = 7$–10 is typical
- Linearity baked in (no breaks)
- Hyper-parameter λ sensitive — needs tuning
- Posterior sampling can be slow on long runs

BVAR pairs naturally with DFM: both deliver smooth, theory-coherent nowcasts to balance the more flexible ML estimators.

Part IV — MIDAS regression

The mixed-data sampling problem, sharply stated

Setup:
We want

$$y_q \;=\; \beta_0 \;+\; \sum_{j=1}^{m} \beta_j \, x_j^{(q)} \;+\; \varepsilon_q$$

where $x_j^{(q)}$ is the j-th monthly observation of indicator x *within* quarter q.

Naive approach:
Stack all monthly lags as free regressors. With L lags and N indicators:

$$\text{params} \;=\; L \times N$$

For $L = 12$ and $N = 8$ that's already **96 free coefficients** with $T_q \approx 60$.

Same OLS overfit story as the BVAR — we need structure.

The MIDAS idea: tie lag weights to a polynomial

Replace L free coefficients with a **smooth polynomial** in lag index:

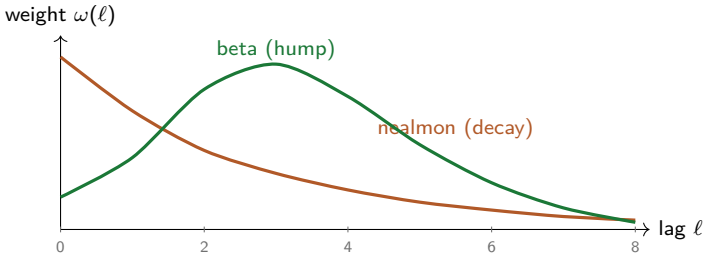
$$y_q \;=\; \beta_0 \;+\; \sum_{j=1}^{N} \beta_j \Big[\sum_{\ell=0}^{L} \omega(\ell; \boldsymbol{\theta}_j)\, x_{j,\,t-\ell} \Big] \;+\; \varepsilon_q$$

Each indicator j now uses only $\dim(\boldsymbol{\theta}_j) = 2$ parameters for L lags — a dramatic dimensionality reduction.

Common weight families:

- **Exponential Almon (nealmon)** — $\omega(\ell) \propto \exp(\theta_1 \ell + \theta_2 \ell^2)$
- **Beta** — $\omega(\ell) \propto \ell^{\theta_1 - 1}(1 - \ell)^{\theta_2 - 1}$
- **Unrestricted (U-MIDAS)** — no polynomial, OK when L is small

What the polynomial weights look like



The same indicator can be told: "weight recent months more" (nealmon) or "the middle of the quarter matters most" (beta).

Estimation: NLS for a smooth loss

The model is *linear in* β but *nonlinear in* $\boldsymbol{\theta}$:

$$\min_{\beta,\boldsymbol{\theta}} \sum_{q} \left(y_q - \beta_0 - \sum_{j} \beta_j \, W_j(\boldsymbol{\theta}_j) \, \mathbf{x}_j^{(q)} \right)^2$$

Solver: non-linear least squares (Gauss–Newton / Levenberg–Marquardt) initialised at

- $\boldsymbol{\theta}_j = (1, -0.5)$ for nealmon (mild decay)
- Plus a few random restarts to avoid local optima

Forecast: once $(\hat{\beta}, \hat{\boldsymbol{\theta}})$ converged, plug in latest monthly $\mathbf{x}$:

$$\hat{y}_{q+1} \;=\; \hat{\beta}_0 + \sum_{j} \hat{\beta}_j \, W_j(\hat{\boldsymbol{\theta}}_j) \, \mathbf{x}_j^{(q+1)}$$

The R code: `midasr`

```r
library(midasr)

# For each indicator x in the spec:
models[[col]] <- midas_r(
  formula = y ~ mls(x, 0:1, 1, nealmon),    # lags 0..1 (monthly-in-quarter)
  start   = list(x = c(1, -0.5))             # nealmon init: gentle decay
)
nowcast <- forecast(models[[col]], newdata = list(x = x_test))$mean
```

- `mls()` = MIDAS lag-structure helper (*months per quarter*)
- Fits one MIDAS per indicator, then aggregates by averaging the nowcasts
- Same output tables as the other estimators

MIDAS: when does it shine?

Best at:

- **Single-variable nowcasts** where lag structure clearly matters
- Capturing release-timing effects (early-month card payments, late-month trade)
- Light: 2–3 parameters per indicator

Limits:

- **No variable selection** — you must pre-pick the indicator
- NLS can converge to a local minimum
- Each indicator runs in isolation — no joint distribution
- Aggregating across indicators is ad-hoc

Solution to the variable-selection problem: ***MIDAS-ML****.*

Part V — MIDAS-ML (sparse-group LASSO)

Why we need MIDAS-ML

With $N \approx 400$ candidate indicators and $L \approx 12$ lags each, even with a 2-parameter polynomial we face

$$\text{candidate regressors} \;\approx\; N \times L \;=\; 4800$$

versus $T_q \approx 50$ quarterly observations.

Two-layer sparsity is natural:

- Many indicators **should not enter the model at all**
- For those that do enter, only **some lags matter**

This is exactly what **sparse-group LASSO** (sg-LASSO) was designed for.

Sparse-group LASSO formulation

Let group g = all lags of indicator g. Minimise

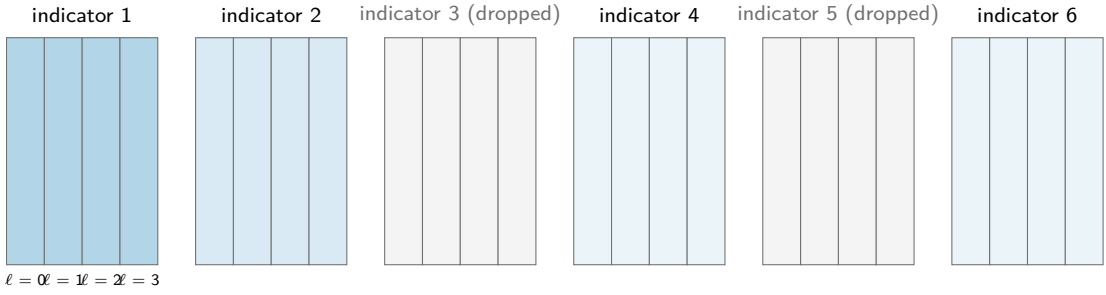
$$\tfrac{1}{2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 \;+\; \lambda\Big[\,\gamma\,\|\boldsymbol{\beta}\|_1 \;+\; (1-\gamma)\sum_g \|\boldsymbol{\beta}_g\|_2\Big]$$

where:
$\|\boldsymbol{\beta}\|_1$ = element-wise LASSO penalty (kills individual lag coefficients); $\sum_g \|\boldsymbol{\beta}_g\|_2$ = group-LASSO penalty (kills entire indicators); $\gamma \in [0,1]$ trades the two off.

Two penalties two effects:

- Within an entering indicator: only a few lags survive
- Across the variable pool: most indicators are dropped entirely

What sg-LASSO does to the design matrix



Group-LASSO kills entire columns of indicators (groups 3 and 5 here). LASSO within surviving groups further darkens only the lags that carry information.

Tuning λ via k-fold CV

Procedure:

1. Build a grid $\lambda_1 > \lambda_2 > \cdots > \lambda_M$ (start tight, loosen progressively)
2. For each λ, split training data into $k = 10$ folds
3. Hold out each fold, fit on the rest, average the validation RMSE
4. Pick $\hat{\lambda}$ at the **minimum CV-RMSE** (or one s.e. above for parsimony)

The γ knob: the `midasml` package treats γ as a user input (default 0.5). Tuning it jointly is rare in practice — a 2D grid blows up.

In practice: `cv.sglfit(..., gamma = 0.5, nfold = 10)`.

The R code: midasml

```r
library(midasml)

# Build the MIDAS design matrix (mls() helpers) + group index
dataset <- prepare_midasml(monthly_X, gdp_q, lags = 12)

# Sparse-group LASSO with 10-fold CV
model <- cv.sglfit(
  x      = dataset$x,
  y      = dataset$y,
  gamma  = 0.5,
  gindex = dataset$gindex,
  nfold  = 10
)
nowcast <- predict(model, newx = dataset$x_test, s = "lambda.min")
```

- Same spec list as MIDAS, but the model picks lags *and* indicators
- Same output tables as the other estimators

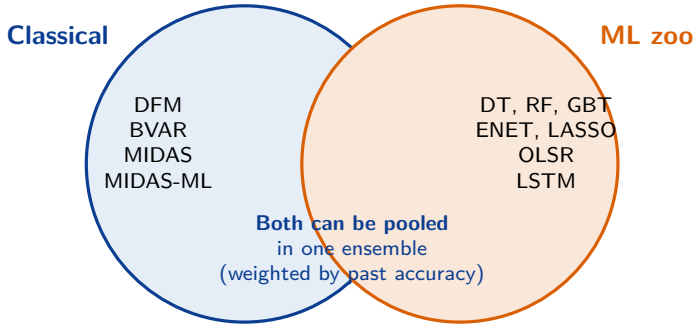
Part VI — Bringing it together

When to reach for which model

Model	Best when...	Watch out for...
DFM	many indicators share one cycle	cycle breaks or multiple unrelated drivers
BVAR	$K \leq 10$ and you want a joint posterior	numerous indicators or strong nonlinearity
MIDAS	a single well-understood indicator (e.g. natural gas production)	high-dim variable selection
MIDAS-ML	hundreds of candidate indicators with sparse signal	low signal-to-noise where shrinkage kills useful coefficients

None of these alone dominates — which is why the production pipeline pools them with the ML zoo.

Complementarity with the ML zoo



- Classical models add **theory-consistent shrinkage** and **coherent uncertainty bands**
- ML zoo adds **nonlinearity** and **variable selection from large pools**
- Weights by past accuracy keep the best of each family in the pooled number

Closing

Four classical workhorses, each answering one question:

- **DFM** — *is there a common cycle?*
- **BVAR** — *what is the joint distribution of GDP and its indicators?*
- **MIDAS** — *how do within-quarter monthly observations weight into the GDP forecast?*
- **MIDAS-ML** — *which of many indicators (and which lags) really matter?*

Thank you.